



# Software Testing and Quality Assurance (05302438)

---

Yadagiri Rama Deepak,  
Assistant Professor  
Computer Science & Applications





## Module-4

### **Black Box & White Box Testing**

- Black Box Testing (Functional testing) - equivalence partitioning, Boundary Value Analysis,
- cause effect graphing,
- Overview of syntax testing,
- White Box Testing (Structural testing) - coverage testing, statement coverage,
- branch and decision coverage,
- Path coverage,
- Comparison of black box and white box testing,
- Case study on black box and white box testing.

## Definition:

**Equivalence Partitioning** is a black box testing technique that divides input data into **valid and invalid partitions** (also called equivalence classes), so **only one test case** from each partition is enough to test the system.

## Goal:

Minimize test cases while ensuring broad input coverage.  
Avoid redundant tests by selecting **representative values**.

## How It Works:

- 1. Identify input conditions** (e.g., input field range, format).
- 2. Divide them into equivalence classes:**
  - **Valid class:** inputs that should be accepted.
  - **Invalid class:** inputs that should be rejected.
- 3. Pick one representative value** from each class for testing.

## Example Scenario:

Suppose you are testing a field that accepts **age between 18 and 60**.

| Partition Type | Range           | Example Test Input |
|----------------|-----------------|--------------------|
| Valid          | 18 to 60        | 25 ✓               |
| Invalid        | Less than 18    | 17 ✗               |
| Invalid        | Greater than 60 | 65 ✗               |

Even though there are infinite values in each range, we only test **one from each partition**.



## Example Use Case in Form Input:

Field: "Password length must be between 8–12 characters"

| Partition | Class      | Example                 |
|-----------|------------|-------------------------|
| Valid     | 8–12 chars | "myp@ssw0rd" ✓          |
| Invalid   | < 8 chars  | "abc123" ✗              |
| Invalid   | > 12 chars | "verylongpassword123" ✗ |



## Benefits:

- Reduces **number of test cases** without compromising quality.
- Easy to understand and implement.
- Great for testing **boundary and mid-range values**.

## Limitations:

- May **miss edge cases** if not combined with **Boundary Value Analysis**.
- Relies on the assumption that all values in a class are processed the same way.

## Black Box Testing (Functional testing)

### 1. Functional Testing

Functional Testing is a type of Software Testing in which the system is tested against the functional requirements and specifications. Functional testing ensures that the requirements or specifications are properly satisfied by the application.





## Black Box Testing (Functional testing)

- This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.
- This testing focuses on checking the user interface, APIs, database, security, client or server application, and functionality of the Application Under Test. Functional testing can be manual or automated. It determines the system's software functional requirements.

## Equivalence partitioning

Equivalence Partitioning Methods is also known as Equivalence Class partitioning (ECP). It is a software testing technique or black-box testing that divides input domain into classes of data, and with the help of these classes of data, test cases can be derived.

- The idea is to partition the input domain of the system into several Equivalence Classes such that each member of the class works similarly.
- If a test case in one class results in some error, other members of the class would also result in the same error.

## Equivalence partitioning

Equivalence Partitioning Methods is also known as Equivalence Class partitioning (ECP). It is a software testing technique or black-box testing that divides input domain into classes of data, and with the help of these classes of data, test cases can be derived.

- The idea is to partition the input domain of the system into several Equivalence Classes such that each member of the class works similarly.
- If a test case in one class results in some error, other members of the class would also result in the same error.

## Boundary Value Analysis

Boundary Value Analysis is based on testing the boundary values of valid and invalid partitions. The behavior at the edge of the equivalence partition is more likely to be incorrect than the behavior within the partition, so boundaries are an area where testing is likely to yield defects.

It checks for the input values near the boundary that have a higher chance of error. Every partition has its maximum and minimum values and these maximum and minimum values are the boundary values of a partition.



## Boundary Value Analysis

In simple terms boundary value Analysis is like testing the edge cases of our software where most of the time it will get broke so it is important to do BVA before deploying the code. There are many other test that are done if you wish to learn them all then you can join our [complete software testing course](#)

## Boundary Value Analysis

### Note:

- A boundary value for a valid partition is a valid boundary value.
- A boundary value for an invalid partition is an invalid boundary value.
- For each variable we check-
  - Minimum value.
  - Just above the minimum.
  - Nominal Value.
  - Just below Max value.
  - Max value.



## Cause Effect Graphing

Black Box Testing **Cause Effect Graphing based technique** is a technique in which a graph is used to represent the situations of combinations of input conditions. The graph is then converted to a decision table to obtain the test cases. Cause-effect graphing technique is used because boundary value analysis and equivalence class partitioning methods do not consider the combinations of input conditions. But since there may be some critical behaviour to be tested when some combinations of input conditions are considered, that is why cause-effect graphing technique is used. **Steps used in deriving test cases using this technique are:**

## Overview of syntax testing:

The syntax testing verifies the grammar, and format of the input data sets that are used in the software either internally or externally. They may be the part of the technical ciphers namely BNF. This type of testing is especially useful to generate test cases which particularly verifies the inputs. It also includes the validation of the code to confirm if they are written as per the programming language syntax, and do not contain inaccurate punctuations and keywords, incorrect usage of brackets etc which can cause unexpected errors like crashes.

## Overview of syntax testing:

The syntax testing involves checking of both the external and internal data sets used in the software with respect to a particular format like the format of the file, schema of database, protocols, and other precise items. Mostly the test cases built on the syntax testing are automated and it involves creation of a large number of test cases.

## Overview of syntax testing:

How to Carry out the Software Syntax Testing?

The software syntax testing is carried out by following the steps listed below –

**Step 1** – Identify the language and its format with which the software is to be compared with.

**Step 2** – Then the syntax of the language is described as explained in the formal notation. Since every input data set follows a specific syntax that may be either officially defined or undocumented.

**Step 3** – The last stage involves validating, and debugging the syntax to confirm that it is uniform, and does not have any flaws. The syntax is generally verified against the below two criterias –

## Overview of syntax testing:

The garbage criteria is the procedure of testing the softwares tolerance with respect to the bad, and filthy data. This criterion is run by feeding inaccurate data sets to the software which do not follow the particular format, and grammar of the syntax.

1. It checks the usual criteria with the help of the various paths of the syntax graphs so that the minimum specifications are satisfied.
2. It checks the garbage criteria with the help of the inaccurate input data sets.

The garbage criteria is the procedure of testing the software's tolerance with respect to the bad, and filthy data. This criterion is run by feeding inaccurate data sets to the software which do not follow the particular format, and grammar of the syntax.

## Overview of syntax testing:

### Advantages of Software Syntax Testing

- The advantages of the software syntax testing are listed below
- The software syntax testing detects defects from the early stages of the software development lifecycle (SDLC).
- The software syntax testing makes the code more maintainable and readable for the developers.
- The syntax testing improves the overall performance, and robustness of the software over a period of time.



## Overview of syntax testing:

### Disadvantages of Software Syntax Testing

- The disadvantages of the software syntax testing are listed below
- The software syntax testing does not cover the standard test cases.
- TThe software syntax testing involves creation of the driver to touch upon all the test cases involving the input data sets.
- TIt is not easy to detect syntax errors, and if undetected they may cause crashing or other problems in the software.

## White Box Testing

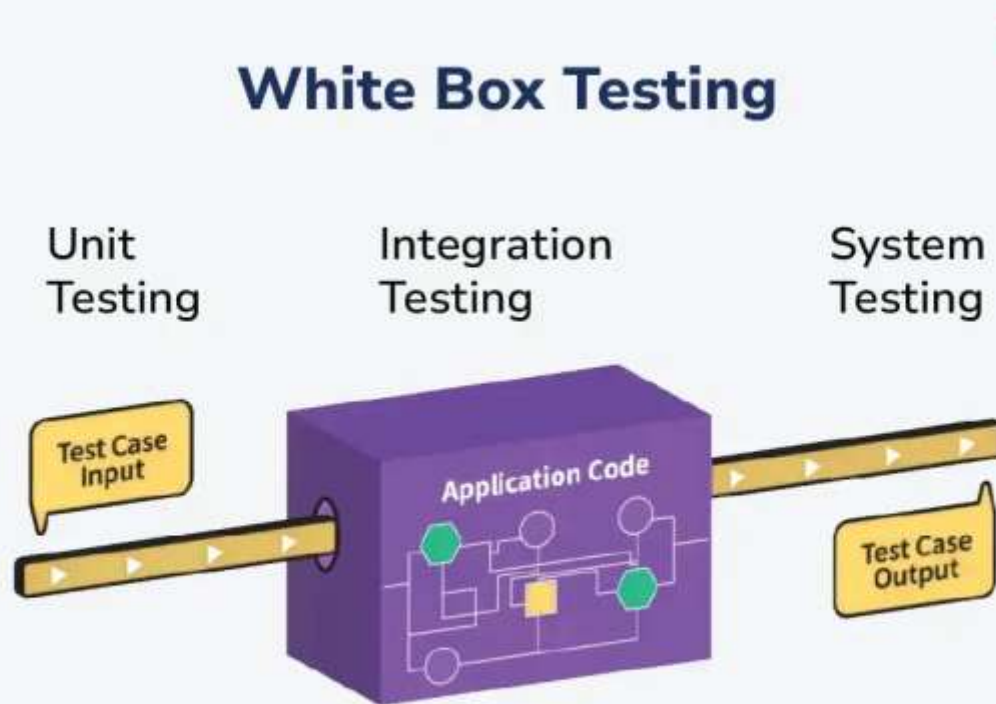
White box testing is a Software Testing Technique that involves testing the internal structure and workings of a Software Application.

The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.

White box testing is also known as Structural Testing or Code-based Testing, and it is used to test the software's internal logic, flow, and structure.

The tester creates test cases to examine the code paths and logic flows to ensure they meet the specified requirements.

## White Box Testing



## White Box Testing

### What Does White Box Testing Focus On?

White-box Testing focuses on the internal workings of an application, ensuring that its logic, structure, and flow operate as intended. Unlike black-box testing, which focuses on user interactions without knowledge of the underlying code, white-box testing involves examining the software's source code directly. Below are the key areas of focus in white-box testing:

## White Box Testing

### 1. Code Logic and Flow

Checks if the program's logic works as intended. This means verifying that code modules (like functions or classes) interact correctly and that control structures such as if-else statements, loops, or switches execute properly. For example, ensuring a login function redirects users correctly based on valid or invalid credentials.

### 2. Code Coverage

Ensures tests exercise as much of the code as possible. This includes:

**Statement coverage:** Every line of code runs at least once.

**Branch coverage:** All decision paths (e.g., true/false conditions) are tested.

**Path coverage:** Every possible route through the code is checked.

This helps find untested or “dead” code that could hide bugs

## White Box Testing

### 3. Data Flow and Variables

Verifies that data is passed and manipulated correctly through the application. This includes ensuring variables are properly initialized, updated, and used without causing any errors or unexpected behavior. Monitoring the flow of data ensures that the system remains stable and reliable as it processes inputs, calculations, and outputs.

### 4. Internal Functions and Methods

Tests the individual functions or methods to ensure they perform their intended tasks accurately and return the expected results. This part of white-box testing focuses on validating business logic, mathematical computations, and other operations within the software. Ensuring these internal processes are correct helps catch potential issues early and improves the reliability of the overall system.

## White Box Testing

### 5. Boundary Conditions

Examines how the code handles edge cases, like the maximum or minimum values for inputs (e.g., a loop running 0 or 100 times, or an input field accepting a 255-character string). This ensures the app doesn't crash at its limits.

### 6. Error Handling and Exception Management

Confirms the program manages errors smoothly, catching exceptions (e.g., invalid inputs) and providing clear feedback. For example, testing if a file upload function handles a missing file gracefully.



## White Box Testing

### Types of White Box Testing:

- Path Testing
- Loop Testing
- Unit Testing
- Mutation Testing
- Integration Testing
- Penetration Testing

# × ○ DIGITAL LEARNING CONTENT



## Parul<sup>®</sup> University



[www.paruluniversity.ac.in](http://www.paruluniversity.ac.in)

